

# Projeto: API de Controle Financeiro Pessoal

## Stack:

- Java
  - Spring Boot
  - Maven
  - H2 Database
  - Postman
- 

## Objetivo do Projeto

Criar uma API REST simples para controle financeiro pessoal, permitindo:

- cadastrar receitas
  - cadastrar despesas
  - listar transações
  - atualizar registros
  - deletar registros
  - calcular saldo
- 

## ETAPA 1 — Criar o projeto Spring Boot

### Acesse:

<https://start.spring.io>

### Configure:

- Project: Maven
- Language: Java
- Spring Boot: versão estável mais recente
- Group: com.helio
- Artifact: finance-api

## Dependências:

- Spring Web
- Spring Data JPA
- H2 Database
- Lombok

## Depois:

- Generate
  - baixar o .zip
  - abrir no IntelliJ
- 

## ETAPA 2 — Estrutura inicial

Crie os pacotes:

- controller
- model
- repository
- service

Sua estrutura ficará assim:

src/main/java/com/helio/financeapi

```
|— controller
|— model
|— repository
|— service
```

---

## ETAPA 3 — Criar a entidade Transaction

### Campos:

- id
- descricao
- valor
- tipo
- data

## Exemplo:

- id: 1
  - descricao: Salário
  - valor: 2500
  - tipo: RECEITA
  - data: 2026-06-02
- 

## ETAPA 4 — Criar enum TipoTransacao

### Valores:

- RECEITA
- DESPESA

Isso ajuda muito na organização do projeto.

---

## ETAPA 5 — Criar Repository

Crie:

TransactionRepository

Extenda:

JpaRepository<Transaction, Long>

Aqui você aprende:

- persistência
  - banco
  - JPA
- 

## ETAPA 6 — Criar Service

### Métodos:

- criarTransacao()

- listarTransacoes()
- buscarPorId()
- deletar()
- atualizar()

Aqui você aprende:

- regra de negócio
  - separação de responsabilidades
- 

## ETAPA 7 — Criar Controller

### Rotas:

#### GET

/transacoes

Lista todas as transações

---

#### POST

/transacoes

Cria uma nova transação

---

#### PUT

/transacoes/{id}

Atualiza transação

---

#### DELETE

/transacoes/{id}

Remove transação

---

## ETAPA 8 — Configurar H2

No application.properties:

- habilitar console H2
- configurar datasource

Depois acessar:

<http://localhost:8080/h2-console>

Isso vai te dar MUITA noção de banco de dados.

---

## ETAPA 9 — Testar no Postman

Testar:

- GET
- POST
- PUT
- DELETE

Exemplo JSON:

```
{  
  "descricao": "Freelance",  
  "valor": 500,  
  "tipo": "RECEITA"  
}
```

---

## ETAPA 10 — Funcionalidade extra (IMPORTANTE)

Criar endpoint:

/saldo

Ele:

- soma receitas
- subtrai despesas

- retorna saldo final

Aqui começa a ficar interessante de verdade.

---

## ETAPA 11 — Melhorar organização

Adicionar:

- DTOs
- tratamento de erros
- validações
- mensagens amigáveis

Não precisa fazer tudo agora.  
Mas deixe anotado.

---

## ETAPA 12 — README do GitHub

Adicionar:

- descrição do projeto
  - tecnologias usadas
  - endpoints
  - prints do Postman
  - como executar
- 

## Objetivo REAL desse projeto

Você NÃO está construindo:  
“um sistema financeiro completo”.

Você está praticando:

- APIs REST
- estrutura back-end
- Java
- Spring Boot
- banco de dados

- organização de projeto
- arquitetura básica

E sinceramente?

Esse tipo de projeto já parece MUITO mais profissional no GitHub do que 500 exercícios soltos.